Design and Analysis of Algorithms

Module 1-2: Introduction

This is an open educational resource.
Submit fixes, improvements, and new material here.

Welcome back!

Last class was mostly syllabus day, but we still learned important things about what an algorithm *is*.

Let's see how much you can recall. This is a *fake quiz*.

Try doing it with pen and paper.

Remember, this is how you will have to do the real quizzes!

Fake Quiz

Q1: Is this procedure an algorithm? Why or why not?

```
unsigned factorial(unsigned n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Q2: What is a small change you could make to change that.

I.e., if it is an algorithm, to make it not one; if not, to make it one.

Intended answers

```
unsigned factorial(unsigned n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Yes, this is an algorithm. Let's consider the important points:

- It definitely terminates. We get closer to the base case (i.e., make progress) with every recursive call.
- Each operation is well defined by the C standard, including overflow.
- There is one input, and the output clearly depends on it.
- We know how to do each operation and each can be carried out in finite time.

Intended answers (2)

How would we change it to not be an algorithm?

There are many answers. Consider this one:

```
unsigned factorial(unsigned n) {
    return n * factorial(n - 1);
}
```

I removed the base case. Now it makes a recursive call unconditionally.

There are some tricky ways to answer this wrongly, though...

Surprising wrong answer

Suppose we did this instead?

```
unsigned factorial(unsigned n) {
    if (n == 1) return 1;  // now it's == instead of <=
    return n * factorial(n - 1);
}
```

- Would `factorial(0)` terminate?
- Actually, it would. `factorial(n - 1)` would become `factorial(0xFFFFFFFF)`. This is defined behavior. Eventually it would work its way back down to n == 1 and terminate.
- However, signed overflow is actually undefined behavior, which is why we made it 'unsigned'

Surprising wrong answer (2)

So it's actually still an algorithm.

But wait it gives the wrong answer! And also, it doesn't work for n > 12! It overflows!

Believe it or not: that doesn't make it stop being an algorithm.
Algorithms can be incorrect.

However, if we made the unsigned int a signed int, then it would not be an algorithm, because every operation would not be defined. In the C standard, signed overflow is undefined behavior.

Some reassurance

Don't worry, you aren't expected to memorize the C standard.

However, you are expected to have a coherent hypothesis of what will happen when you do something, and be able to explain it.

I.e., "if I subtract 1, it will wrap around when it hits zero" is one reasonable hypothesis. "It will crash" is another reasonable hypothesis.

If you don't have such a hypothesis, then the operation becomes undefined behavior. That's the kind of thing that will lose you points.

Questions?

Standard-based learning

Like I mentioned before, this class will follow a standard-based learning approach.

That means that there will be certain skills that you are expected to master, and you will be given multiple attempts to demonstrate mastery of each one.

There are 8: 4 before the midterm and 4 after.

Let's learn what these areas will be!

Before the midterm

1. You will be able to understand Bachmann-Landau notation (i.e., Big-O, Big-Θ, small-o, etc.). You will be able to demonstrate this understanding by proving that algorithms have a given runtime bound.
2. You will be able to implement and analyze a self-balancing tree, specifically a scapegoat tree, modify it to fit problem-specific constraints, and compare it to the performance of other data structures.
3. You will be able to design and implement a custom sorting algorithm that exceeds the speed of standard sorting algorithms on data with certain characteristics.
4. You will learn about complexity categories, such as P, NP, and NP-complete. You will learn how to determine and prove which category a problem belongs to.

After the midterm

5. You will be able to identify when a greedy algorithm is appropriate and design one.
6. You will be able to recognize when pre-computation is appropriate, and will demonstate the *prefix-sums* technique.
7. You will be able to recognize when to use full dynamic programming, and to solve a dynamic programming problem with a reasonable technique (such as memoization).
8. You will be able to work with graphs. You will demonstrate the algorithm design technique of "graph modelling", and implement an algorithm using this technique.

Short modules

In addition, there are some shorter modules that cover interesting information you haven't seen in other classes, such as:

1. How to make a blazingly fast memory-allocator, and the issues with garbage collection and less structured allocators like malloc and free.
2. How b-trees work, why they're so fast, and why they are Rust's sorted dictionary.
3. Some basic number theory, so you can solve a common kind of technical interview and competitive programming problem.

These aren't specifically measured by standards, but they might show up in the other problems, and they are still important to learn.

Remember the name of this class...

It's not "algorithms" or "algorithms and data structures".

It's *design* and *analysis* of algorithms.

We will be making custom algorithms to solve specific problems. We will be applying tools of mathematical analysis on these algorithms to prove that they have certain properties.

So this class is a little creative and a little mathy.

Don't be afraid: my hope is that proofs (especially inductive ones) will really *click* in this class. They will be easier and shorter than you think!

Questions?

The Language

In the past, I allowed any language.

The point was the algorithms, not the specific language used to implement them.

Besides, it is a fun growth opportunity for students who found the class easy to try learning a new language.

But with some experience teaching, my perspective changed a bit.

Systems languages

I realized that you cannot separate analysis of an algorithm from the fundamental time and space constraints of the system it runs on.

We'll see an example later of how the choice of programming language completely changes the mathematical properties of the runtime performance.

The issue is that application languages, especially those with garbage collection and immutable strings, hide some very important performance considerations.

Therefore, I made a rule that the language had to be a system language. Specifically, C, C++, or Rust, with special permission for other systems languages (like Zig or Odin).

What is a systems language?

What makes a language a *systems language*?

[What do you think?]

What a systems language is

It's a language that was mainly designed for writing *system software*.

E.g., things like drivers, the OS kernel, and in general, things that need to be fast.

How does that influence the design of the language?

I.e., [what are some things that systems languages have in common?]

Properties of systems languages

- They usually don't have a runtime.
- [What's a runtime and [why does, say, *Python* have one but not *Rust*]?
- If you need to write an OS kernel, you can't rely on other programs, including a runtime.
- Who would load it? The kernel that was written in a language that requires a runtime?
- At some point there needs to be a raw executable that just *runs* without needing any other software to be present.

Properties of systems languages (2)

- They usually do not have garbage collectors, or if they do, it can be turned off.
- [Why would that matter?]
- Garbage collectors typically require a runtime. Otherwise, they bloat the executable. They also create unpredictable performance characteristics.
- Either the garbage collector stops the world, or it runs in other threads that need to lock and synchronize.
- Therefore, GCs add a *non-trivial* amount of performance overhead.

Properties of systems languages (3)

Systems languages are usually *fast*.

That does not mean that every program written in a systems language will be faster than one written in an applications language. It's possible to write slow code in a fast language (especially when more complex data structures like strings are involved)

Systems languages give you *control*.

You typically only pay for what you use. The language does not add features that have a performance impact unless you want it.

The value of systems languages

Therefore, using a systems language is very useful.

You will be forced to think about everything the computer is doing.

You will be forced to make performance decisions at a reasonably low level, although still not the lowest... (*laughs in assembly*)

Which systems language?

As I said, in the past I allowed many.

The problem is, some choices still allowed performance bugs to hide.

For example, C++ and Rust allow you to write very slow code without realizing it.

Realizing it requires experience. Why do we always pass strings by reference in C++? Why do we use AsRef<str> in Rust? If you don't know, you might write code that is slower than Java or Python in so-called "fast" languages.

These are not things that you will pick up on your own; they would have to be covered in class. Valuable info, but something that really should be in an systems class.

The language will be C

We will use C for this class.

Only C. No exceptions.

C is very low level. It does not have any built-in data structures except for arrays.

Every expensive operation requires a function call. There will be no surprises. It will actually make the analysis easier to know, e.g., when a string copy is happening vs. a pointer assignment.

We are going to make sure we understand our algorithms at both high and low levels.

Besides, more practice with pointers and manual memory management won't hurt.

Questions?

Why study algorithms to this level?

Haven't they already created the algorithms for us?

Lots of algorithms have been created. Many more will be created in the future.

But it's rare that you can just use an algorithm without *any* customization at all.

That would imply that the whole problem is just a function call away.

Why study algorithms to this level? (2)

You are being trained to be a computer scientist, not just a programmer.

A computer scientist is a mix between a mathematician and an engineer. They are skilled at problem solving in the abstract.

You aren't just here to implement someone else's algorithm. You are meant to learn the field deeply enough to be able to make your own.

And not just make them, but prove that they are suitable for a given purpose.

An example

To show you why these skills are important, I want to show you a version of an algorithm that I stumbled across in the industry.

It has a serious bug that isn't obvious, and requires some knowledge of the underlying language and algorithmic analysis to see.

However, it's simple enough to understand if those problems are pointed out.

First, let me set the stage by explaining what this software was meant to do...

The report generator

Briefly, I worked at a chemical plant as an automation engineer.

There was a push to improve monitoring of alarms and anomalous events, so engineers were asked to generate reports about them.

The report would have a list of instruments and the number of alarms and other events that occurred in a given month.

To avoid having to manually compile this report from the database every month, it was important that the process be automated.

How it was supposed to work

The generator was supposed to do the following:

- allow the user to enter a date range
- for each instrument, query the database to determine the number of alarms, warnings, and notifications it had generated in that range.
- then, generate a table summarizing this information as a CSV file*.

Very straightforward. It could be implemented straightforwardly as an Excel macro.

* CSV means "comma separated value. It's a simple spereadsheet data format where each cell's data is separated by a comma. It's supported by Excel and allows you to generate spreadsheets without needing a complicated DOCX library. I strongly recommend learning it if you haven't yet.

My instructions

I was told to speak to the engineer at another unit. They already had a report generator.

The engineer was happy to show it off. We sat down to talk. They started the report generation process and we had a long conversation about other things while we waited...

About 30 minutes later, it occurred to me that the process should have finished by now...

I was told, "no, it will be another 90 minutes or so. Let's grab lunch!"

Hang on a second...

Generating some reports should not take 2 whole hours.

It's a nice feature that it gave me an excuse to get lunch, but that was not intentional.

I asked to take a look at the source code to figure out what was taking so long. The engineer who maintained it (a different one) was somewhat offended, but they gave in.

What I'm about to show you is a small, explanatory snippet, written in Java* designed to express how it worked.

Try your best to find the performance bug!

* The program was originally written writen in a Janky proprietary language that no one uses anymore.**

** Most of the software in the plant was written in and for janky proprietary systems. We used like 3 different versions of Unix across different units

The code

```
public static String generateReportCsv(Date start, Date end) {
    InstrumentEvents[] instrumentEvents =
        Db.queryInstrumentEvents(start, end);

    String result = "";
    for (var inst : instrumentEvents) {
        result += inst.name + ",";
        result += inst.nAlarms + ",";
        result += inst.nWarnings + ",";
        result += inst.nNotifications + ",";
        result += ",,"; // double commas mean end of row
    }

    return result;
}
```

Try to find what could be making this extremely slow.

The bug

The reason I picked Java is that it has the same string semantics as the proprietary language the report generator was originally written in.

Specifically, strings are immutable.

There are good reasons to make strings be immutable.

However, it poses a problem when we use the "+=" operator...

Concatenating strings

Just because the operator for concatenating strings is `+`, it does not mean that it is as fast as integer addition.

This operator is *overloaded*. It does *different things* depending on its operands.

In most languages, concatenating strings takes *linear time*, whereas adding numbers takes *constant time*.*

We will learn/review what these terms mean, but suffice to say, constant time means it always takes the same amount of time (usually fast), while linear time means the time it takes is proportional to the length of at least one of the strings.

This is true of Java, too. Concatenating strings is linear time.

* Fun fact: in Erlang, it takes constant time. How could they manage that? If we have time, we can talk about "ropes".

Allocating strings

Strings are variable length data, and they can depend on information outside the program. Therefore, they are typically dynamically allocated.

When you concatenate two dynamically allocated strings, the following things happen:

- First, the system counts how long the combined string will be. This is fast in Java.
- Then, a new string is allocated that is big enough to hold the combined string.
- Then, the characters are copied from the first string into the new string.
- Finally, the characters are copied from the second string into the new string.

Representing this time mathematically

Intuitively, this means that the time it takes to concatenate strings is something like this:

$$T = (n + m) \times k$$

Where T is the total time taken

n and m are the lengths of the first and second string respectively

k is the average amount of time it takes to copy one character*

* it's the average because sometimes it's faster or slower. For example, you can load at least 16 ascii characters into one SIMD register in many modern CPUs. However, if the string has 17 characters, that last character ends up requiring a whole register move just for that one character. So the exact time per character goes from being 1/16 of a SIMD op to a whole SIMD op.

Computing the time taken

Knowing that, let's look at our code again. Assume k = 1 for simplicity:

```
String result = "";
for (var inst : instrumentEvents) {
    result += inst.name + ",";  // T = len(result) + len(name) + 1
    result += inst.nAlarms + ",";
    // T = len(result) + 1 + len(nAlarms) + 1
    result += inst.nWarnings + ",";
    // T = len(result) + 1 + len(nWarnings) + 1
    result += inst.nNotifications + ",";
    // T = len(result) + 1 + len(nNotifications) + 1
    result += ",,"; // T = len(result) + 2
}
```

See the problem now? Pay attention to $len(result)$.

The problem

Result keeps getting longer!

Suppose all the instrument strings are 5 letters long.

The first concat is $T = 0 + 5 + 1$
Then $T = 6 + 5 + 1$
Then $T = 12 + 5 + 1$
Then $T = 18 + 5 + 1$
Finally $T = 24 + 2 = 26$

So how much time does the first iteration of the loop take?

The problem (2)

$$T_{iter1} = 6 + 12 + 18 + 24 + 26 = 86$$

But what about iter 2? It's different, because it's still adding to the same result. The length **starts** at 26:

$$T_{iter2} =$$
$$(26 + 6) + (26 + 6 \times 2) + (26 + 6 \times 3) + (26 + 6 \times 4) + (26 + 6 \times 4 + 2) = 216$$

And then?

$$T_{iter3} =$$
$$(52 + 6) + (52 + 6 \times 2) + (52 + 6 \times 3) + (52 + 6 \times 4) + (52 + 6 \times 4 + 2) = 346$$

$$T_{iter4} =$$
$$(78 + 6) + (78 + 6 \times 2) + (78 + 6 \times 3) + (78 + 6 \times 4) + (78 + 6 \times 4 + 2) = 476$$

The problem (3)

Each iteration takes 130 time units longer than the one before.

What's the total time? Say we have I instruments.

Try writing a math expression for T_{total}

Total time

$$T_{total} = \sum_{i=2}^{I}(130i) = 130 \times \sum_{i=1}^{I} i = 130 \times \frac{i(i+1)}{2}$$

I recall something like 10,000 instruments. How many time units is that?

$$T_{total} = \frac{130 \times 10{,}000(10{,}000+1)}{2} \approx 6.5 \text{ billion units}$$

If each unit took a microsecond (because it was running in a slowly interpreted proprietary language on a slow computer), that's about 108 minutes.

* We're using the identity that $\sum_{i=1}^{i} = \frac{i*(i+1)}{2}$

The fundamental problem

Notable software engineer and stack-overflow co-creator Joel Spolsky calls this kind of algorithm a *Shlemiel the Painter* algorithm.

Shlemiel is the guy from the joke about a road painter who gets slower and slower each day. When his boss asks why, he complains that the paint can gets farther away each time he paints a stretch.

Here, we're starting over and copying the whole string every iteration of the loop. It's like we're wandering all the way back to the paint can.

Carrying the paint can

So how do we do something more sensible?

No matter what, if there are 10,000 instruments and 6 bytes of data per instrument, we expect to copy 60,000 bytes.

However, we don't have to re-copy the whole old string over again. That was the problem.

So what if we just made one really big string all at once? Then we could copy all the smaller strings into it without re-allocating and copying a new string.

How to fix it

```
var result = new StringBuilder(inst.length * 26);
for (var inst : instrumentEvents) {
    result.append(inst.name).append(',')
          .append(inst.nAlarms).append(',')
          .append(inst.nWarnings).append(',')
          .append(inst.nNotifications).append(',')
          .append(",,");
}

return result.toString();
```

What did that do?

StringBuilder is like string, but it has extra space after it. We can decide how much space it starts out with.

It does one, big allocation at the beginning.

Those appends just copy each string into the existing buffer without re-allocating and without copying the entire string, just the new one.

So now the time used is reasonable.

Questions?

Conclusion

This is a standards-based class. It's important to know what the standards are.

We will be using C, because knowing every time we might be invoking an expensive operation is valuable.

Knowing how to use algorithms is not enough. As computer scientists, we are expected to go deeper.

We need to really understand what's going on under the hood. Sometimes it has a dramatic impact on performance.

Unworked Practice Problems

1. Add comments to the new, StringBuilder-based code to express how much time each line takes.
2. Calculate the estimated total time using the same assumptions: 10,000 instruments, 1 op per byte, 1 microsecond per op.
3. In java, it would likely take 1 nanosecond per op or less. How long would that have taken? Can you imagine how there might not be much of an impetus to fix it, even though it's a serious performance error?
4. Why is it worth fixing anyway? What if everyone had the attitude that "good enough" was fine, even if they were using vastly more time than needed.